

Notes on Multi-Tier Geospatial Embeddings

Ideas for combining hierarchical structures, behavioral dynamics, and macro-to-micro regional context using Open Source tools.

1. Multi-Pipeline Framework for Location Embeddings

When modeling geographic entities (Country $\rightarrow$ City $\rightarrow$ Postal Code), treating locations as simple categorical IDs fails to capture their spatial, hierarchical, and socioeconomic relationships. To provide a production-grade representation, we employ a **multi-pipeline** architecture that synthesizes three distinct embeddings of geographic/location "truth":

I. Neighborhood Physical Vibe (E_{local})

- **Concept:** Uses **Uber H3 (Resolution 8)** indexing and **OpenStreetMap (OSM)** feature engineering to capture the physical "vibe" or signature of the land.
- **Function:** Infer wealth and behavior via **socioeconomic proxies** (e.g., counting luxury retail, transit hubs, and building types).
- **Best For:** High-frequency updates (daily ingestion) and hyper-local context like walkability and infrastructure friction.

II. Directly Surveyed Socioeconomic (E_{census})

- **Concept:** Joins explicit government demographic tables (**US Census/Eurostat**) with **Overture Maps** administrative boundaries.
- **Function:** Wealth and behavior statistics from **official socioeconomic sources**.
- **Best For:** Structural grounding, provides the verified, reported official statistics (income, age, education)

III. Regional Behavior (E_{macro})

- **Concept:** Leverages pre-trained **Geographic Foundation Models** (e.g., Clay, SatCLIP) mapped to **DMA/MSA** boundaries via County FIPS.
- **Function:** Captures **regional market** demographics and behavior.
- **Best For:** Distinguishing between physically identical neighborhoods in different economic climates (e.g., a suburb in New York vs. one in Cleveland).

OSM contains data about physical features in a location - the so-called *points of interests* (POI). OSM data is updated frequently but POIs are indirect socioeconomic indicators. On the other hand, a census directly surveys these indicators but it is updated sporadically. To resolve the conflict between "stale ground truth" (Census) and "current but indirect" (OSM), we should not train these features into a single embedding. Instead, we generate independent sub-embeddings E_{local} , E_{census} and E_{macro} , and make these available to downstream neural networks via a **feature store**.

By combining all three sub-embeddings, downstream machine learning systems get a rich geospatial context. Depending on the application, a single sub-embedding or all three can be selected for training the downstream model.

2. Comparing Open-Source Frameworks

Three popular options exist for pairing geospatial data with behavioral/demographic characteristics:

Google's S2Vec / AlphaEarth Framework

- **Mechanism:** Self-supervised Masked Autoencoders trained on satellite images and built environments.
- **Pros:** Phenomenal zero-shot predictions for socioeconomic indicators (e.g., median income). Very straightforward.
- **Cons:** Cloud API dependencies (Google Earth Engine), black-box vectors (poor explainability), and visual biases.

Uber H3 + OpenStreetMap (OSM) Pipelines

- **Mechanism:** Mathematical indexing (H3) combined with feature engineering on crowdsourced map metadata.
- **Pros:** 100% offline, lightning-fast on local infrastructure, and allows complete custom selection of input features.
- **Cons:** Requires a self-hosted data pipeline (e.g., Spatial Representations for AI - SRAI toolkit).

Overture Maps + Census Data (open data alternative)

- **Mechanism:** Joining explicit government demographic tables (US Census/Eurostat) into Overture administrative boundaries, compressed via an Autoencoder or PCA.
- **Pros:** Highly accurate human/economic truth. Transparent and explainable.
- **Cons:** Fractured global consistency; census metrics vary significantly by nation.

MSA + RANGE (open data alternative)

- **Mechanism:** Open-source foundation embeddings for U.S. Metropolitan Statistical Areas (MSAs) are actively maintained by academic labs and geospatial consortia, such as RANGE (Retrieval Augmented Neural Fields for Multi-Resolution Geo-Embeddings).
- **Pros:** Pre-trained model, hosted directly on Hugging Face.
- **Cons:** USA-only, based on MSAs (not DMAs).

Uber H3 + OSM is a solid alternative and the recommended approach for E_{local} . Overture Maps + Census Data is an open data alternative that is suitable for E_{census} . We can use MSA + RANGE for E_{macro} if we compute the mapping MSAs $\leftrightarrow$ DMAs.

3. Uber H3 + OpenStreetMap Approach

This option has major advantages:

- Global Consistency: OSM data is structured globally
- Offline/On-Premise: Can run 100% locally on our server
- Open License: Apache 2.0 License
- Customization: We can pick the real-world elements on a map that matter to us

The last point deserves clarification:

In OpenStreetMap (OSM), every physical thing on Earth is tagged with open text metadata (e.g., `amenity=cafe`, `building=apartments`, `highway=primary`). Because one controls the input features, one gets to choose what behavior the embeddings capture. For example:

- **For a FinTech / Fraud model:** Explicitly configure the embedding pipeline to extract features like the count of banks, ATMs, luxury stores, and casinos within a specific H3 hexagon. The resulting embedding represents "financial behavior and commercial density."
- **For a Logistics / Delivery model:** Filter the input features to only care about warehouse zones, traffic lights, primary roads, and apartment complex gates. The resulting embedding represents "delivery friction and infrastructure layout."

The H3 Grid (Local Memory Hierarchy)

Instead of messing with complex latitude and longitude polygons for countries, cities, and postal codes, H3 divides the entire world into millions of nested hexagons.

H3: Hexagonal, Hierarchical, Hexadecimal

Because it is purely mathematical, it runs on our own server inside an open-source Python library. No 3rd-party APIs are required.

Country → City → Postal Code hierarchy:

Every hexagon on earth has a unique, short string ID (e.g., `85283473ffffffff`). Because the index is hierarchical, one can truncate or change the "resolution" of the index instantly in local memory:

- **Resolution 4 (Macro / DMA / Country level):** A massive hexagon covering an entire metropolitan market.
- **Resolution 6 (City level):** A medium hexagon capturing an urban center.
- **Resolution 8 (Hyper-Local / Postal Code level):** A tight hexagon covering a neighborhood.

To find out if a specific postal code hexagon belongs to a specific city hexagon requires but a simple check of the H3 ID string. It is not necessary to do a heavy geographic database query.

Local Deployment

We can build this solution using open-source tools without relying on a 3rd-party service. A standard approach is to use a Python framework called **SRAI (Spatial Representations for Artificial Intelligence)**.

An production-ready pipeline works in three simple steps:

Step A: The Local Data Ingestion

A compressed open-source data dump of the world (or target countries) is downloaded from a site like **GeoFabrik** (which provides free, daily updated OpenStreetMap files in `.pbf` format). This data is loaded into our server or local machine.

Step B: The Hexagon Mapping (The Hierarchy)

We index our dataset using the open-source `h3` Python library to index. We map every Postal Code and City boundary to its corresponding H3 hexadecimal IDs.

Step C: Train a Local Embedding (Hex2Vec)

We pass filtered features (the custom POIs) and our H3 grid into an open-source embedding model like **Hex2Vec** (built directly into the SRAI toolkit).

The model uses a simple neural network to look at the hexagons, see what features are inside them, see which hexagons are neighbors, and output a compact, 64-dimensional vector for every hexagon on earth.

Customization of Features

Everything in the OpenStreetMap (OSM) and Overture Maps is crowdsourced and standardized by global committees, and **virtually every meaningful physical object on Earth is mapped and categorized**. When building a custom embedding, one can choose features from tens of thousands of options. To keep a model clean, these features are structured into a few massive, high-level buckets.

The primary POI (Point of Interest) categories and raw features available are the following:

Commercial & Economic Activity

This is what most people think of when they hear "points of interest." Overture Maps manages over 64 million global business points categorized into a standardized taxonomy:

- **Socioeconomic Proxies:** Banks, ATMs, luxury retail (`brand=gucci`), car dealerships, and pawn shops.
- **Sustenance & Culture:** Cafes, fast-food chains (`brand=mcdonalds`), fine dining, bars, and clubs.
- **Services:** Gyms, dry cleaners, barbers, and gas stations.
- **Industry & Commerce:** Warehouses, industrial parks, offices, and logistics hubs.

Example: A postal code with 50 `cafe` and `boutique` POIs creates an embedding vector representing a high-density, high-disposable-income urban neighborhood.

Infrastructure & Real Estate

OSM doesn't just map points; it maps polygons and shapes. One can extract features based on what kind of buildings physically dominate an H3 hexagon.

- **Residential Types:** Apartments, detached single-family homes, student dormitories, or hotels.
- **Building Attributes:** The open-source data often includes `building:levels` (how tall a building is) and `building:flats` (how many apartments are inside).
- **Industrial vs. Green:** The percentage of land area tagged as `landuse=industrial` vs. `landuse=grass` or `amenity=park`.

Mobility, Transit, & Traffic (The Flow Features)

Mobility POIs are important in embeddings used for logistics, risk, or delivery time estimation.

- **Transit Hubs:** Subway stations, bus stops, train platforms, and airports.
- **Road Network Density:** Ratio of `highway=motorway` (freeways) vs. `highway=residential` (local streets).

- **Friction Points:** The number of traffic lights, stop signs, speed bumps, and pedestrian crosswalks in a zone.

Civic, Demographic, & Safety Anchors

These features are heavily used to establish the cultural and societal baseline of a geographic region.

- **Education & Health:** Schools, universities, kindergartens, hospitals, pharmacies, and clinics.
- **Demographic Proxies:** Places of worship (churches, mosques, synagogues—highly predictive of regional demographics), community centers, and public libraries.
- **Safety & Government:** Police stations, fire stations, town halls, and courthouses.

4. Local Embedding Pipeline

The local server can inspect an H3 hexagon, count all the relevant/chosen items or POIs present, and build a simple *feature matrix (or histogram)* before compressing it into an embedding. Like this:

```
[ Some Hexagon ] —> POIs Counts —> [ Feature Vector ] —> ...
    - banks: 3
    - apartments: 14
    - traffic_lights: 2
    - schools: 1

... [ Feature Vector ] —> [ NN / Encoder ] —> [ Dense Embedding ]
                                     (64 dimensions)
                                     straightforward math
```

What is filtered through before training dictates the nature of the embedding. For example, filtering for transit and roads yields a **logistics vector**, while filtering for retail brands and financial institutions yields an **economic/behavioral vector**. And so on.

H3 Resolution 8 is the most appropriate for our goals

Resolution 8 is a good balance between human behavior and the geometry of administrative boundaries:

- A Resolution 8 hexagon matches how humans naturally experience neighborhoods (e.g., "are there cafes within walking distance of my apartment?")
- An average suburban or urban zip code will contain between 10 to 50 Resolution 8 hexagons. It is large enough to show structural variations (e.g., "this zip code has a commercial strip on its east side and residential homes on its west side"), but small enough to keep computational costs down.
- At 0.737 km^2 , even moderately suburban hexagons will contain at least a few POIs

Interpretation

When two vectors are "similar" under the **Hex2Vec** approach, it indicates the locations share a **behavioral archetype** rather than a coordinate:

1. Similar vectors represent hexagons that are **"functional twins"**. Based on the POI filters, they generate identical "economic gravity centers" (e.g., two luxury retail hubs) or "operational friction" (e.g., two

- logistics bottlenecks) regardless of their physical separation.
2. Similarity implies **similar local context**. Because the encoder considers neighboring hexagons, two vectors are close only if they share both internal features and a similar surrounding environment (e.g., "residential blocks adjacent to green space").
 3. In attention-based models, matched locations activate the same **learned query**. Similarity means both areas triggered the same latent "aspect" (see below).

Scaling to postal code boundaries

To scale the above hexagon-to-embedding pipeline up to arbitrary postal code boundaries, we have to combine or aggregate these individual hexagon vectors into a single, representative vector for the postal code. Let us assume that we know how to determine which H3 hexagons lie inside a postal code boundary.

A simple aggregation approach is by **mean pooling**, which is just the vector average of all the hexagon embeddings within the postal code. This is extremely fast, zero extra parameters to train, and mathematically stable. But an obvious flaw is that it treats quiet, empty hexagons the same as dense, active ones.

A better approach is **weighted pooling**, where we weight each hexagon by its "human activity level" (e.g., POI count or population). This forces the postal code embedding to represent where the actual "action" is. If 90% of the businesses in a zip code are clustered in 2 hexagons, those 2 hexagons will dominate the postal code's vector.

A more elegant approach is by attention-based pooling. We will probably not implement this anytime soon, but it is the "deep learning way". For completeness sake, this approach is described next.

Hexagon-to-Postal Attention Pooling

In this approach, the hexagon embeddings are fed into a lightweight Attention layer (similar to the pooling layer in a Transformer). The network learns a query vector representing "What makes an hexagon relevant to our business task?" and automatically weights the hexagons within a postal code.

In the downstream neural network model, the training data consists of examples where the input is a Postal Code (associated with a list of its constituent hexagon embeddings) and the target is some Business Outcome. To understand what attention pooling actually achieves let us first review the single-query case.

Single-query case

Inside the attention pooling layer, the model randomly initializes a trainable vector called the **"query vector."** This query vector is the model's evolving definition of: *"What is the ideal prototype of a highly relevant hexagon for this task?"*

When a batch of postal codes passes through the model during training, the attention mechanism performs the following steps for each individual postal code:

- Step A: The model compares the embedding of every hexagon in that postal code against the trainable query vector. It calculates a similarity score for each hexagon. If a hexagon's physical features match what the query vector is currently "looking for," it gets a higher raw score.
- Step B: Because one postal code might contain 5 hexagons and another might contain 50, we cannot use raw scores. The model takes all the raw similarity scores for a single postal code and runs them through a math function (Softmax) that scales them. This forces all the weights within that specific

postal code to sum to exactly 1.0 (or 100%), regardless of how many hexagons are inside. A 5-hexagon ZIP code might split its weights [0.8, 0.1, 0.05, 0.03, 0.02], while a 50-hexagon ZIP code splits its weights across 50 values.

- Step C: The model multiplies each hexagon's embedding by its newly calculated percentage weight and sums them together. The result is a single, fixed-size Postal Code Embedding that has highlighted the most "important" hexagons.

This final, fused postal code vector is passed into the rest of the downstream neural network model to make the final Business Outcome prediction.

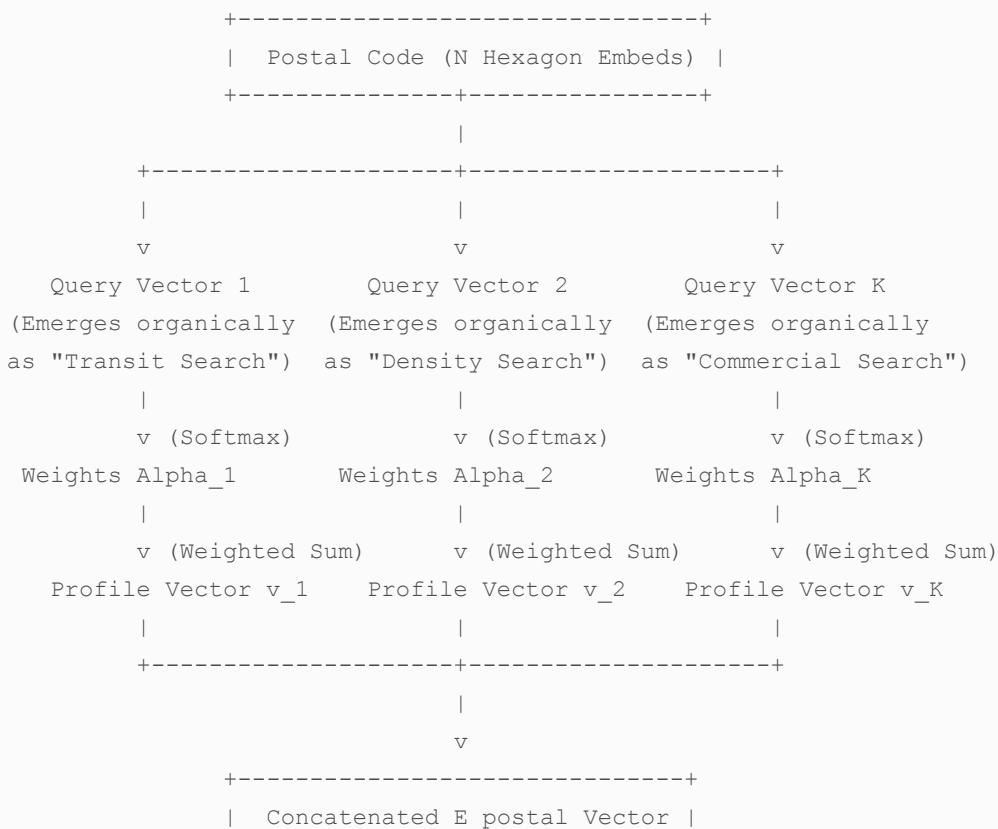
What's is being learned through "attention"

You can think of the single query vector as a global search term or a constant question that the model is asking every hexagon on Earth:

- Query: *"Are you an < economically active transport hub >?"* (With "economically active transport hub" being the abstract concept that was learned by training the values of the query vector).
- Every hexagon in a postal code is compared to this same query.
- Hexagons that match this query get high weights; the ones that don't get low weights.

Multiple query vectors

Instead of training a single query vector $\mathbf{q}$, we train a query matrix $\mathbf{Q}$ containing K different query vectors (often called "heads" in attention literature, or "aspects"). **Crucially, we do not program or tell the model what these query vectors should look for.** We do not label $\mathbf{q}_1$ as "transit" or $\mathbf{q}_2$ as "residential." At initialization, they are purely random numbers.



```
| [ v_1 || v_2 || ... || v_K ] |
+-----+
```

Emergent semantics

During training, the downstream task drives the optimization. To minimize prediction error, the backpropagation process forces the query vectors to **spontaneously differentiate** and specialize in searching for different geographical archetypes:

1. **Automatic Role Assignment:** If the Business Outcome requires (say) predicting delivery times which involves knowing both road speeds and drop-off times, the math of gradient descent will drive $\mathbf{q}_1$ to adapt its weights until it acts as a filter that highlights highway/infrastructure hexagons. Simultaneously, it will drive $\mathbf{q}_2$ to become a filter that highlights high-density apartment blocks.
2. **Dynamic Weights:** For any given postal code, each member hexagon h_i is compared to these self-discovered queries. The network computes K independent sets of weights (normalized via Softmax to sum to 100% across that specific postal code's hexagons):

$$\mathbf{v}_k = \sum_{i=1}^N \alpha_{i,k} \mathbf{E}_{h_i} \quad \text{where} \quad \alpha_{i,k} = \text{Softmax}\left(\frac{\mathbf{q}_k \cdot \mathbf{E}_{h_i}}{\sqrt{D}}\right)$$

3. **Multi-Faceted:** The final postal code embedding ($\mathbf{E}_{\text{postal}}$) is a concatenation of these specialized views:

$$\mathbf{E}_{\text{postal}} = [\mathbf{v}_1 \parallel \mathbf{v}_2 \parallel \dots \parallel \mathbf{v}_K]$$

If a postal code is a suburban commuter town, the sub-vector $\mathbf{v}_1$ (transit) might indicate high accessibility, while $\mathbf{v}_3$ (commercial) indicates near-zero retail presence. The model has mapped the *nuance* of the neighborhood without a human ever writing a rule.

5. Integrating Surveyed Socioeconomic (Census)

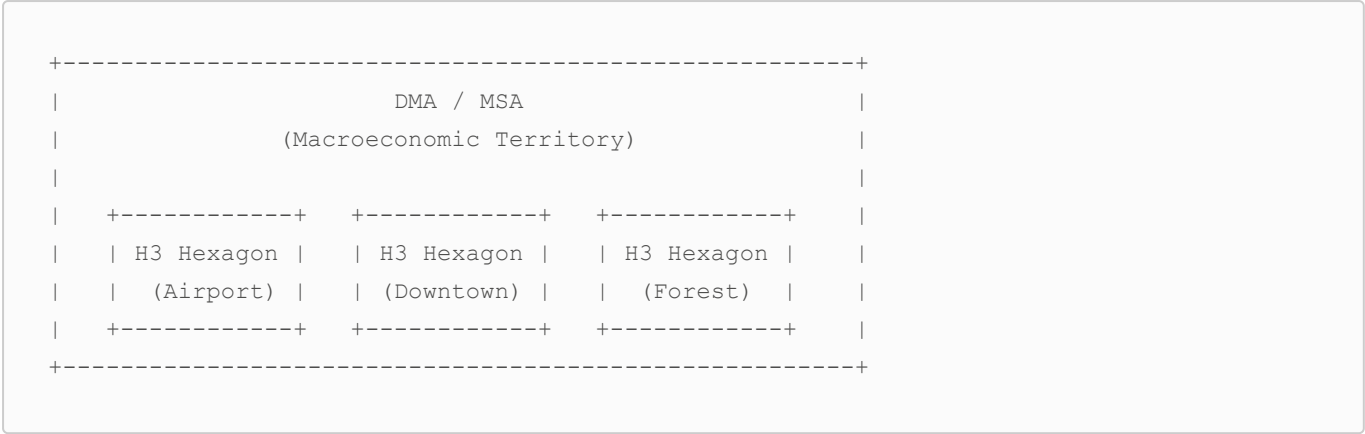
[PENDING]

6. Integrating Regional Behavior (DMAs and MSAs)

Designated Market Areas (DMAs) represent media and consumer culture gravity, while Metropolitan Statistical Areas (MSAs) reflect commuting patterns and economic engines. Both add macro context that hyper-local grids miss.

DMA is Macro, H3 is Micro

- **H3 Resolution 8/9:** Microscopic (~ 0.1 to 0.74 km^2). Finely subdivides Earth into runways, parks, downtown street blocks, etc.
- **DMA:** Macroscopic ($\sim 45,000 \text{ km}^2$). Groupings of entire counties matching media markets.
- *Reality:* Tens of thousands of H3 hexagons fit into a single DMA.



Leveraging Pre-Trained MSA Embeddings

Rather than training regional vectors from scratch, we instead leverage pre-trained geographic foundation models specialized in socioeconomic and urban signals.

- **Providers:** Open-source foundation embeddings for U.S. Metropolitan Statistical Areas (MSAs) are actively maintained by academic labs and geospatial consortia. Leading weights—such as **Clay (Clay Foundation)**, **Prithvi (NASA & IBM)**, and **RANGE (Multi-Scale Geo-Embeddings)**—are hosted directly on **Hugging Face** (e.g., `microsoft/SatCLIP` or `mvrl/RANGE-database`).
- **Licensing:** These pre-trained models are published under highly permissive open-source licenses, typically **Apache 2.0** or **MIT**, allowing for fully self-hosted, commercial production use without external call quotas.
- **What they capture:** These vectors are trained using contrastive learning or masked autoencoders over multi-temporal satellite data (Sentinel-2) and census indicators, capturing structural density, economic development, and macro-urban features of the MSA.

DMA-MSA Joining through County FIPS Codes

Because DMAs and MSAs are both built by grouping whole counties together, we can map them dynamically using a **County FIPS Codes** instead of heavy spatial geometry overlaps:

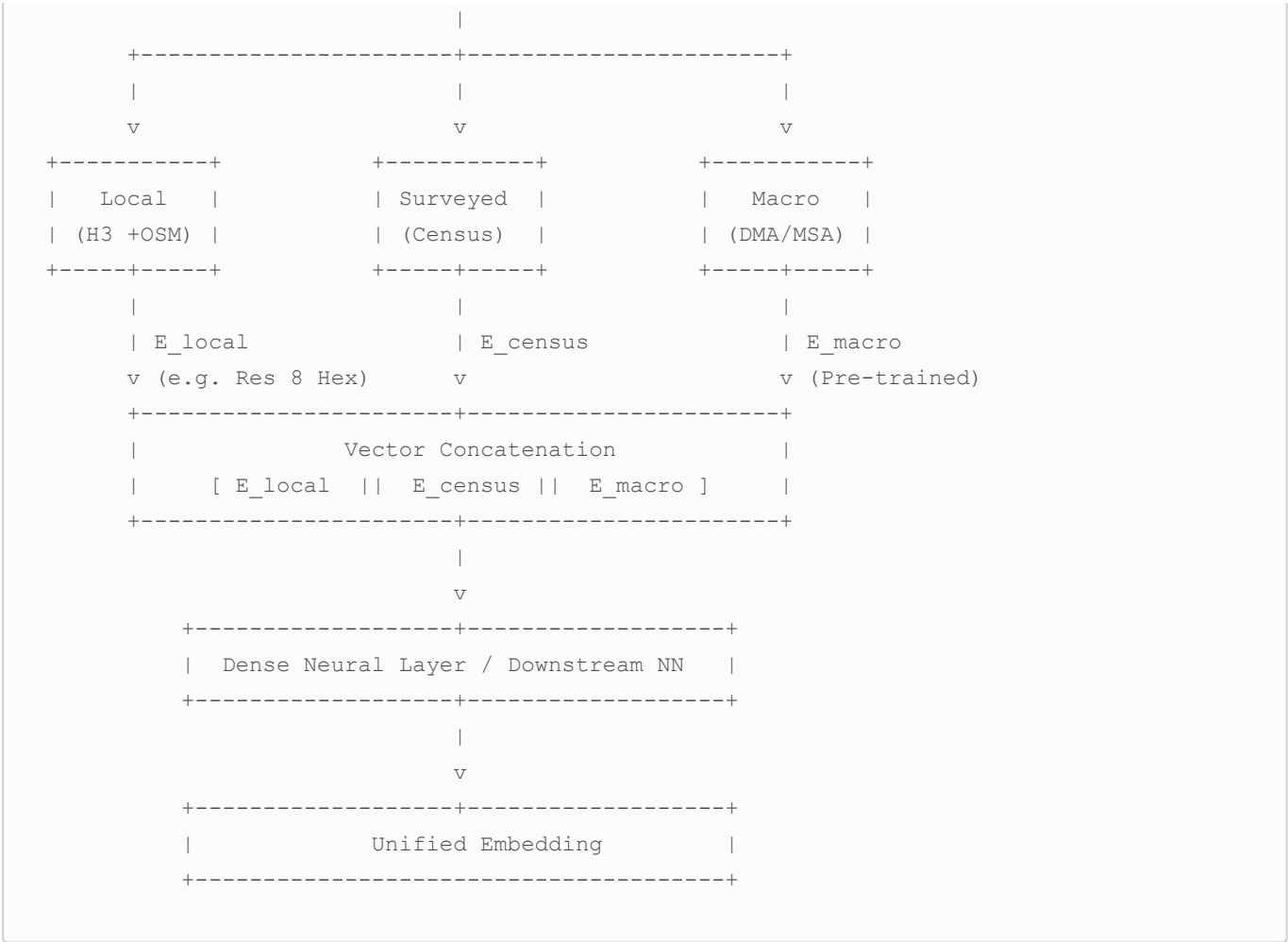
DMA ← [County FIPS Code] → MSA

1. **Source Mapping:** Pull official annual US Census *Statistical Area Delineation Files* (County → MSA) and open academic/NBER crosswalks (County → DMA).
2. **Population Weighting:** Because a DMA contains multiple counties and potentially multiple MSAs, use county populations to perform a population-weighted merge or vector pool to compute a weighted macro-vector.

7. Multi-Pipeline Architecture

To prevent spatial redundancy and eliminate behavioral blind spots (e.g., an identical-looking suburban H3 hexagon in New York vs. Cleveland), implement a **Multi-Pipeline Fusion Architecture**:





1. Local Spatial Vector ($\mathbf{E}_{\text{local}}$)

- **Source:** Uber H3 (Res 8/9) + OpenStreetMap Points of Interest (POIs).
- **Features:** Density, walkability, specific infrastructure types (e.g., counts of cafes, apartments, traffic lights).
- **Focus:** Hyper-local physical context.

2. Surveyed Socioeconomic Vector ($\mathbf{E}_{\text{census}}$)

- **Source:** US Census / Eurostat demographic tables joined to Overture Maps administrative boundaries.
- **Features:** Median income, age distribution, educational attainment, population density, and household composition.
- **Focus:** Structural socioeconomic grounding via verified, officially reported statistics.

3. Regional Behavioral Vector ($\mathbf{E}_{\text{macro}}$)

- **Source:** Pre-trained MSA Embeddings via DMA Crosswalk.
- **Features:** Average regional income, GDP gravity, media market, network connectivity.
- **Focus:** Macroeconomic scaling multiplier.